

# OOP (Object Oriented Programming)

→ Constructors &c.

\*

1.) Creating a structure:-

```
struct student {  
    string name;  
    int roll_no;  
    int semester;
```

```
} s1, s2;  
student s1;  
int main() {  
    student s1;
```

```
class student {  
    string name;  
    int roll_no;  
    int semester;
```

```
} s1, s2;  
student s1;  
int main() {  
    student s1;
```

→ Incapsulation i.e. combine member functions & data members

- 4

→ Adding functionality :- (To members)

```
struct student {
```

```
    ~~~~ ;  
    ~~~~ ;  
    ~~~~ ;
```

→ data members

```
void display
```

(default values) of  $\hookrightarrow$  initialize variable  $\hookrightarrow$   $\hookrightarrow$  Constructors  
1) Constructors main difference  $\hookrightarrow$  struct 1st class  $\hookrightarrow$   
-  $\hookrightarrow$   $\hookrightarrow$   $\hookrightarrow$   $\hookrightarrow$  destructors

$\hookrightarrow$  Forming a constructor:-

$\Rightarrow$  "student ()" (default).

$\Rightarrow$  student (string name, int roll-no, int semester)  
{  
    name = "Zain";  
    roll-no = 05;  
    semester = 2;  
} (specified).

$\Rightarrow$  student () {  
    name = "X";  
    roll-no = 0;  
    semester = 1;  
} (default).

Calling:-

$\Rightarrow$  s1 ();  $\rightarrow$  default constructor runs.  
 $\Rightarrow$  s1 ("Y", 1, 3)  $\rightarrow$  specified.

\* OOP ki life a last stage destructors.  
ko run karna ha.

(Storing the records of 100 students)

```
=> int main() {  
    student s1, s2, s3, s[100];  
    s1.name = "ali";  
    s1.roll-no = 5;  
    s1.semester = 6;  
}
```

```
void Read() {  
    for (int i=0; i<100; i++) {  
        cin >> s[i].name;  
        cin >> s[i].roll-no;  
        cin >> s[i].semester;  
    }  
}
```

Assigning values of variables in class & struct ←  
This is & function & one, This is & constructor & ←  
— One, This is assign & Manually res & one

~~struct student~~ (~~st~~)

~~struct~~

```
student Results() {  
    student temp;  
    return temp;  
}
```

default

specified.

- \* struct ka function.
- \* Constructor wala function.
- \* Int main() function.

→ student result function ko call kia

→ temp say constructor ko call kia

## ~ OOP - Class ~

```
class Student{  
    public:  
        int Rollnumber;  
        string name;  
        float gpa;  
};
```

```
int main(){  
    Student s1;
```

Main difference  
of class and struct  
is access specifies.

- 1.) Private
- 2.) Public
- 3.) Protected

\* Private → class me agar  
inheritance use kia  
wa ha to app data  
members ko access  
kr skty hein.

\* Public ko app  
class ky bahar  
kahin b use  
kr skty hein.

\* Protected ky use  
ky liye bi  
inheritance ka  
hona zroori ha.

\* class objects  
ky liye blueprint  
ha or data  
members by default  
private hoty hn.

```
class Student {
```

```
    --- ;
```

```
    --- ;
```

```
    --- ;
```

```
Student Student (int a, string n, float g) {
```

```
    Roll NO. = 5;
```

```
    Name = "Ali";
```

```
    gpa = 3.8;
```

```
}
```

```
Student () {
```

```
    Roll NO = 6;
```

```
    Name = "Ahmad";
```

```
    gpa = 3.66;
```

```
}
```

```
int main () {
```

```
    Student s1 (5, "Ali", 3.99);
```

```
    Student s2; (default constructor call hoga)
```

\* Kisi bhi oop ky

program me sab sy

phly jo program

object ki life

ma execute hota

ha wo constructor

hota ha.

\* constructor obj

ko initialize karta

ha.

\* Function overload

ing ka matlab

asa function jis

ma same names

do different

parameters ky

lye use hon.

\* "Setter function" is

lye use hota ha

ky app us object

ko apni need ky

according values

assign kr skty

hein.

Setter Function inside a class:-

```
*void write (int a, string b, float c) {  
    Roll_no = 5 a;  
    Name = "ABC" b;  
    GPA = 3.99 c;  
}
```

```
void Read () {
```

```
    cout << " Roll Number : " << Roll_no << endl;  
    cout << " Name : " << Name << endl;  
    cout << " GPA : " << GPA << endl;
```

```
}
```

```
} ; // class yahan end hogai ha.
```

```
int main () {
```

```
    Student s1;
```

```
    s1.write(5, "Ali", 3.98);
```

```
    s1.Read();
```

```
    return 0;
```

```
}
```

## ~ OOP - LAB ~

→ Creating a class of "Myclass":-

```
class MyClass {  
    public: (Access specifier)  
        int myNum;  
        string Name;  
};
```

Object is an instance of a class.

"::" scope resolution operator

```
int main() {  
    MyClass myObject;  
    myObject.myNum = 90;  
    myObject.Name = "Ali";
```

```
    cout << myObject.myNum << endl;  
    cout << myObject.Name << endl;
```

```
    return 0;
```

```
}
```

(Creating Functions)

- 1.) Inside class
- 2.) Outside class

```
class MyClass {  
    public:  
        void myMethod();  
};
```

void MyClass :: myMethod ( ) ;

#

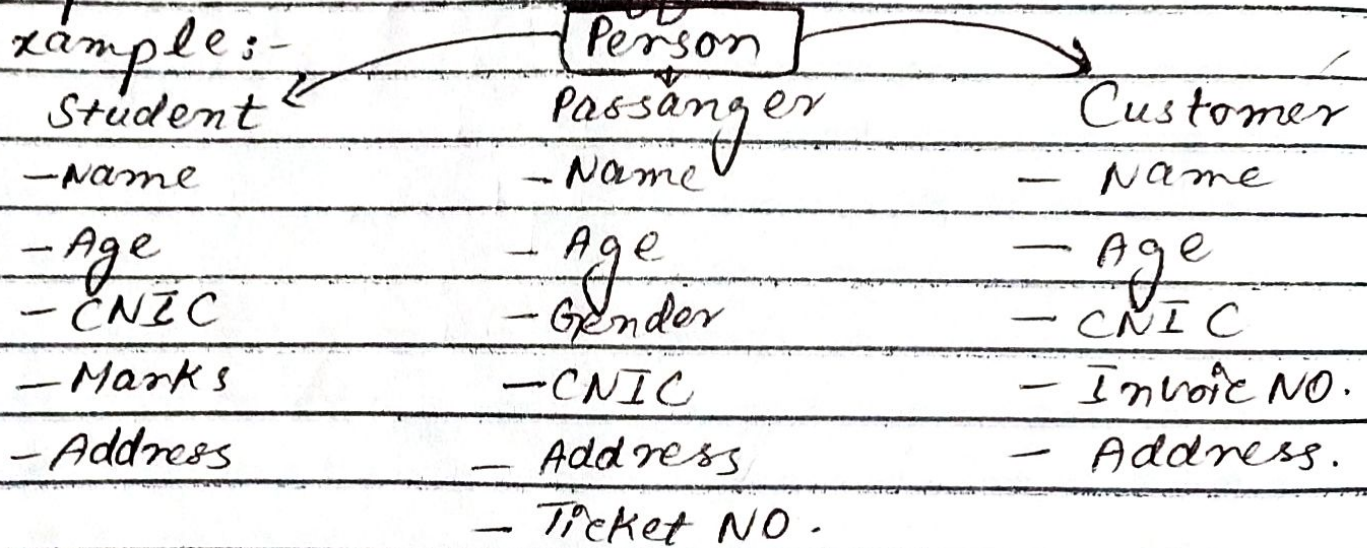
## Lecture 2 (OOP)

### Inheritance

→ Agar function me overwriting nahi hogi to inheritance achieve nahi hogi or uski wajah sy polymorphism b achieve nahi hogi.

→ Inheritance me aik parent hota and or baqi uski child hongy. Jo information jo base class me hogi wo child class me b lazmi hogi but implementation different ho sakti ha.

→ Example:-



class Person {

public:

visibility mode.

Person

base class

```
int age;
int cnic;
string n;
string add;

write();
display();
```

→ using access specifier child class me base class sy attributes functions ko inherit kar lia.

→ child class

→ Agar hamary pass base class ky attributes public hen to child class ky object sy base class ky attributes ko access kar skty hen.

→ Agar private hon to nahi kar skty.

→ Jab b hum child class ka object bnayn gy to phly parent class a constructor call hoga then child class ka call hoga.  
~ & Destructor ~

→ Last program that is called by an object is known as a destructor.

→ Destructor phly hamesha child class ka call hoga then base class a call hoga.

~ & Virtual Functions ~

→ Jab hum base class ma virtual function bnayn gy to phir usky baad hum uska object nahi bnay skty or secondly wo virtual function har child class ma mojood hoga.

! Lab - OOP

Constructor :- (Special case ka function)

- Constructor ka name wahi hoga jo class ka name hoga
- Constructor object ko bnaty hi sab sy phly call hony wla program hota ha.
- Iski koi return datatype pe nahi hoti.
- Ye empty constructor b ho skta ha or parameterized b ho skta ha isky alawa aik or "copy constructor" b hota ha jis ma ham aik object ki properties ko dusry object ma copy kr skty hen.

\* Use karny ki wajah:-

```
class Student { //class
```

```
    { Student() { } //constructor (default)
```

```
    { Student(int a) { } //parameterized constructor
```

```
int main() {
```

```
    Student s1; // default cons ... call hoga.
```

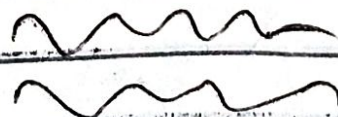
```
    Student s1(2); // parameterized call hoga.
```

~~~~~  
In ka use tab hota jab hum setter functions ma kisi attribute to value assign karna bhool jyn

to usy automatically initialize  
ya set karne ky liye cons-  
tructor ka use hota ha jis sy  
hum early initialization kar skty  
hen.

→ Jab hum kisi class ma constructor  
nhi bnaty to by default wahan  
par aik empty constructor ban  
jata ha or agar compiler b  
constructor na bnaye to phir  
kabi b object create nai ho  
skty. Ye wala constructor bnta  
hi object bnane ky liye hota  
ha.

→ Jab hum constructor bana lety  
hen to phir compiler khud  
sy constructor nhi bnata ku  
ky ab hamary banaye gye cons-  
tructor sy object ban skta  
ha.



```
#include<iostream>
using namespace std;
```

```
class Student {
```

```
private:
```

```
    string name;
```

```
    int age;
```

```
    int RollNO;
```

```
    string adress;
```

```
public:
```

```
    Student(string name, int a, int r, string ad) {
```

```
        this->name = n;
```

```
        age = a;
```

```
        RollNO = r;
```

```
        adress = ad;
```

```
    };
```

```
int main() {
```

```
    Student s1("Zain", 19, 05, "MZD");
```

```
    return 0;
```

```
}
```

ID = 2020  
Name = Not assigned  
Salary = 0.0

displayDetails();

```
class Employee {  
    private: int ID;  
             string name;  
             float salary;
```

```
public:
```

```
Employee() { int ID; string name; float salary;
```

```
    ID = 0;  
    name = "Not Assigned";  
    salary = 0.0;  
}
```

```
void displayDetails() {  
    cout << "Employee Details" << endl;  
    cout << "ID : " << ID << endl;  
    cout << "Name : " << name << endl;  
    cout << "Salary : " << salary << endl;  
}
```

```
};
```

```
int main() {  
    Employee e1;  
    e1.displayDetails();  
    return 0;  
}
```

show details .

= PK 123456 .

= Zain .

= 50,000 .

```
class BankAccount {
```

```
private:
```

```
    string AccountNO;
```

```
    string AccountHolder;
```

```
    int balance;
```

```
public:
```

```
    BankAccount(string AccountNO, string AccountHolder, int balance) {
```

```
        this->AccountNO = AccountNO;
```

```
        this->AccountHolder = AccountHolder;
```

```
        this->balance = balance;
```

```
    }
```

```
void showAccountDetails() {
```

```
    cout << "Account Details " << endl;
```

```
    cout << "Account Number : " << AccountNO << endl;
```

```
    cout << "Account Holder : " << AccountHolder << endl;
```

```
    cout << "Account Balance : " << balance << endl;
```

```
}
```

```
};
```

```
int main() {
```

```
    BankAccount b1("PK123456", "Zain", 50,000);
```

```
    b1.showAccountDetails();
```

```
    return 0;
```

```
}
```

$$w = l = 1.0$$

$$l \neq w$$

Area - Function  
display - Area

```
class Rectangle {
```

```
private:
```

```
    float length;
```

```
    float width;
```

```
public:
```

```
    Rectangle () {
```

```
        length = 1.0;
```

```
        width = 1.0;
```

```
    }
```

```
    Rectangle (float l, float w) {
```

```
        length = l;
```

```
        width = w;
```

```
    }
```

```
    Rectangle (float l) {
```

```
        length = l;
```

```
        width = l;
```

```
    }
```

```
    float Area () {
```

```
        return length * width;
```

```
    }
```

```
    void displayArea () {
```

```
        cout << "Area : " << Area() << endl;
```

```
    }
```

class base {  
public:  
};

~~class child : public base {~~

};

int main() {

Rectangle r1;

r1.Area();

r1.displayArea();

Rectangle r2(2.0, 3.1);

r2.Area();

r2.displayArea();

Rectangle r3(3.6);

r3.Area();

r3.displayArea(); 4

return 0;

}